

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

по Лабораторной работе № 5

«Процедуры, функции, триггеры в PostgreSQL»

по дисциплине «Проектирование и реализация баз данных»

Вариант свой. БД «Легендарный парк»

Обучающийся: Ашур Амир Куссаевич

Факультет: прикладной информатики

Группа: К3239

Направление подготовки: 09.03.03 Прикладная информатика

Образовательная программа: Мобильные и сетевые технологии 2024

Преподаватель: Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург, 2026

1. Цель работы

Овладеть практическими навыками создания и использования хранимых процедур, функций и триггеров в базе данных PostgreSQL.

2. Практическое задание

Вариант 2.2. Выполнить следующее:

- Создать 3 хранимые процедуры для индивидуальной БД «Легендарный парк» (3 балла).
- Создать 5 оригинальных триггеров для индивидуальной БД.

3. Выполнение

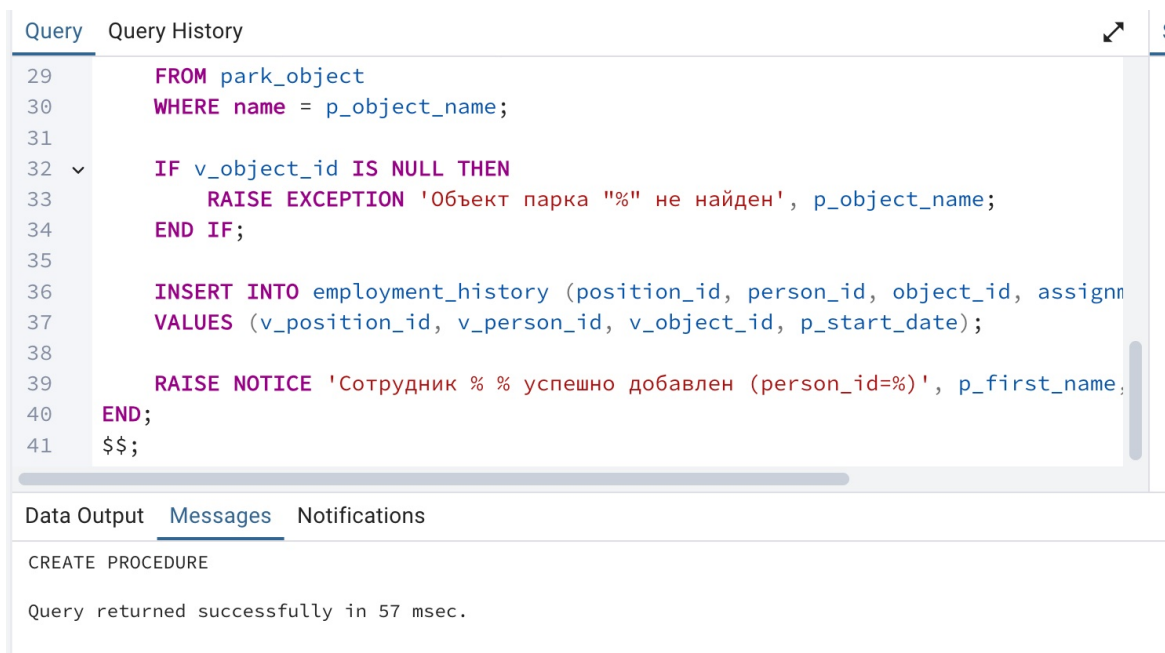
3.1. Хранимые процедуры

Хранимая процедура — объект базы данных, представляющий набор SQL-инструкций, компилируемый один раз и хранящийся на сервере. В отличие от функций, процедуры не возвращают значение напрямую, однако могут выдавать данные через параметры INOUT или через RAISE NOTICE. Вызываются командой CALL.

3.1.1. Процедура add_employee — добавление нового сотрудника

Процедура принимает ФИО, название должности, название объекта парка и дату назначения. Внутри она добавляет человека в таблицу person, находит нужные ID должности и объекта по их названиям и записывает строку в таблицу employment_history. Если должность или объект не найдены — выбрасывает исключение.

Создание процедуры выполнено успешно — сообщение CREATE PROCEDURE подтверждает регистрацию объекта в БД.



```
Query  Query History
29      FROM park_object
30      WHERE name = p_object_name;
31
32  IF v_object_id IS NULL THEN
33      RAISE EXCEPTION 'Объект парка "%" не найден', p_object_name;
34  END IF;
35
36  INSERT INTO employment_history (position_id, person_id, object_id, assignn
37  VALUES (v_position_id, v_person_id, v_object_id, p_start_date);
38
39  RAISE NOTICE 'Сотрудник % % успешно добавлен (person_id=%)', p_first_name,
40  END;
41  $$;
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 57 msec.

Рисунок 1 — Создание процедуры add_employee

Тест корректного вызова: процедура успешно добавила сотрудника Сергея Кузнецова. В Messages выведено NOTICE с подтверждением и присвоенным person_id=5.

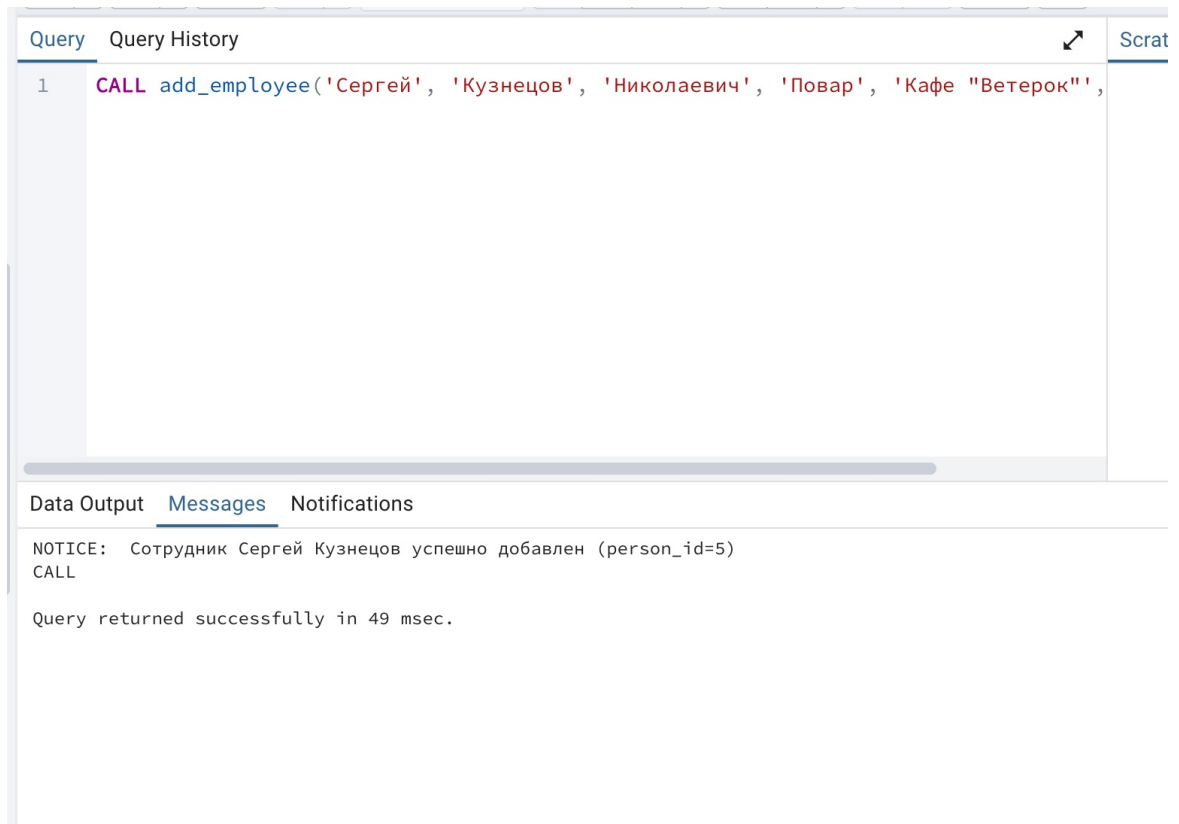


Рисунок 2 — Успешное добавление сотрудника через процедуру `add_employee`

Тест ошибочного вызова: при передаче несуществующей должности процедура корректно выбросила исключение, прервав транзакцию и обеспечив целостность данных.

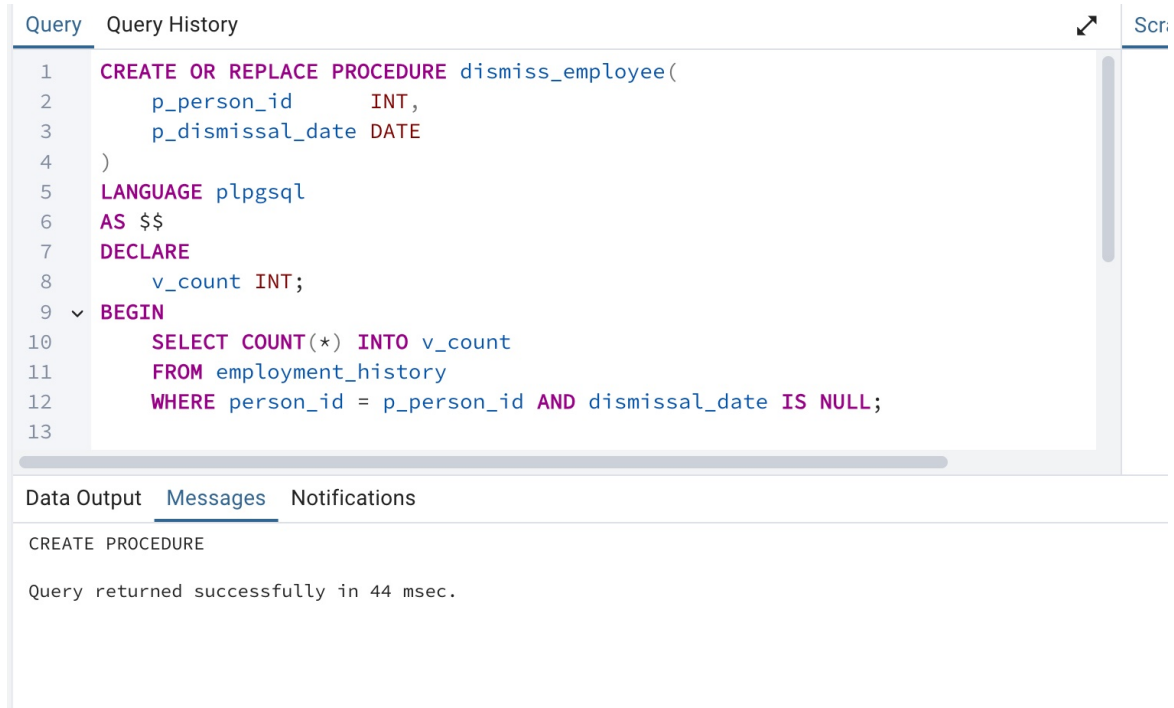


Рисунок 3 — Процедура `add_employee` блокирует добавление с несуществующей должностью

3.1.2. Процедура dismiss_employee — увольнение сотрудника

Процедура принимает person_id сотрудника и дату увольнения. Проверяет наличие активной записи в employment_history (dismissal_date IS NULL). Если активный сотрудник найден — проставляет дату увольнения. Если нет — выбрасывает исключение.

Создание процедуры:



The screenshot shows a database IDE interface. The top pane is titled 'Query' and contains the following SQL code:

```
1 CREATE OR REPLACE PROCEDURE dismiss_employee(  
2     p_person_id      INT,  
3     p_dismissal_date DATE  
4 )  
5 LANGUAGE plpgsql  
6 AS $$  
7 DECLARE  
8     v_count INT;  
9 BEGIN  
10    SELECT COUNT(*) INTO v_count  
11    FROM employment_history  
12    WHERE person_id = p_person_id AND dismissal_date IS NULL;  
13
```

The bottom pane is titled 'Messages' and shows the following output:

```
CREATE PROCEDURE  
  
Query returned successfully in 44 msec.
```

Рисунок 4 — Создание процедуры dismiss_employee

Тест корректного вызова: увольнение ранее добавленного сотрудника (person_id=5) прошло успешно. В лог выведено NOTICE с person_id и датой увольнения.



Рисунок 5 — Успешное увольнение сотрудника через процедуру `dismiss_employee`

Тест ошибочного вызова: при попытке уволить несуществующего сотрудника (`person_id=999`) процедура корректно выбросила исключение.

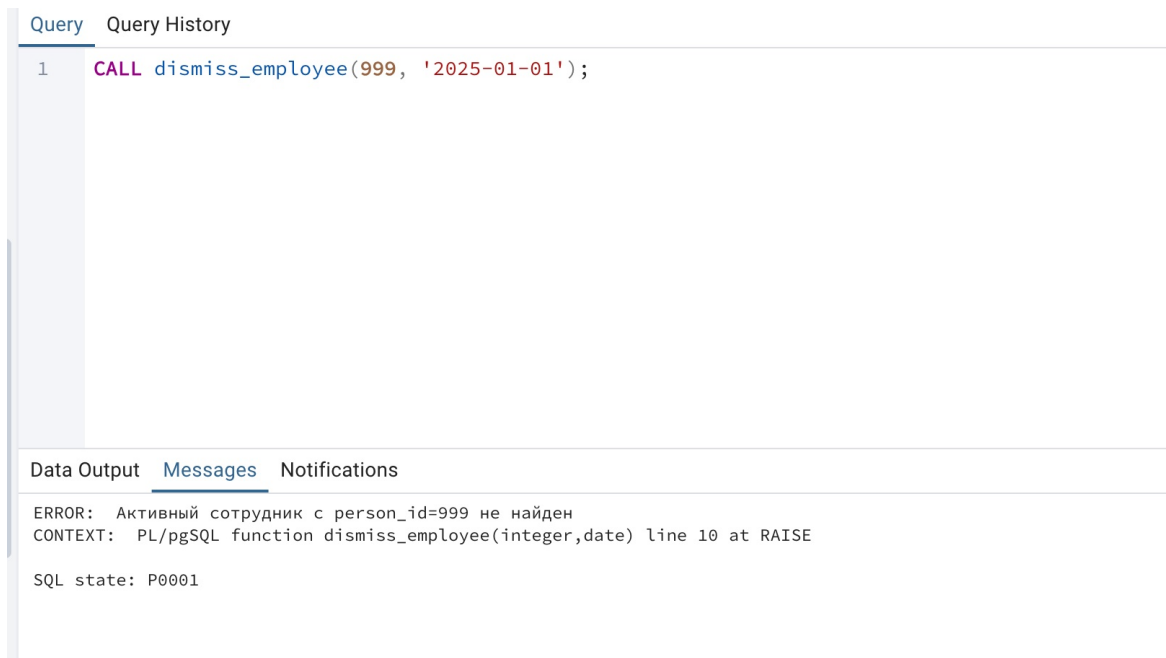


Рисунок 6 — Процедура `dismiss_employee` блокирует увольнение несуществующего сотрудника

3.1.3. Процедура `add_dish_with_price` — добавление блюда с ценой

Процедура добавляет новое блюдо в таблицу `dish` и одновременно устанавливает начальную цену в таблице `dish_price`. Выполняет проверку: категория блюда и объект

парка должны существовать; цена не может быть отрицательной. При нарушении условия выбрасывается исключение.

Создание процедуры:



```
Query Query History
33      RAISE EXCEPTION 'Цена не может быть отрицательной (получено: %)', p_pr
34      END IF;
35
36      INSERT INTO dish (name, description, category_id)
37      VALUES (p_dish_name, p_description, v_category_id)
38      RETURNING dish_id INTO v_dish_id;
39
40      INSERT INTO dish_price (dish_id, object_id, assignment_date, price)
41      VALUES (v_dish_id, v_object_id, p_date, p_price);
42
43      RAISE NOTICE 'Блюдо "%" добавлено (dish_id=%) с ценой % руб.', p_dish_name
44      END;
45      $$;
```

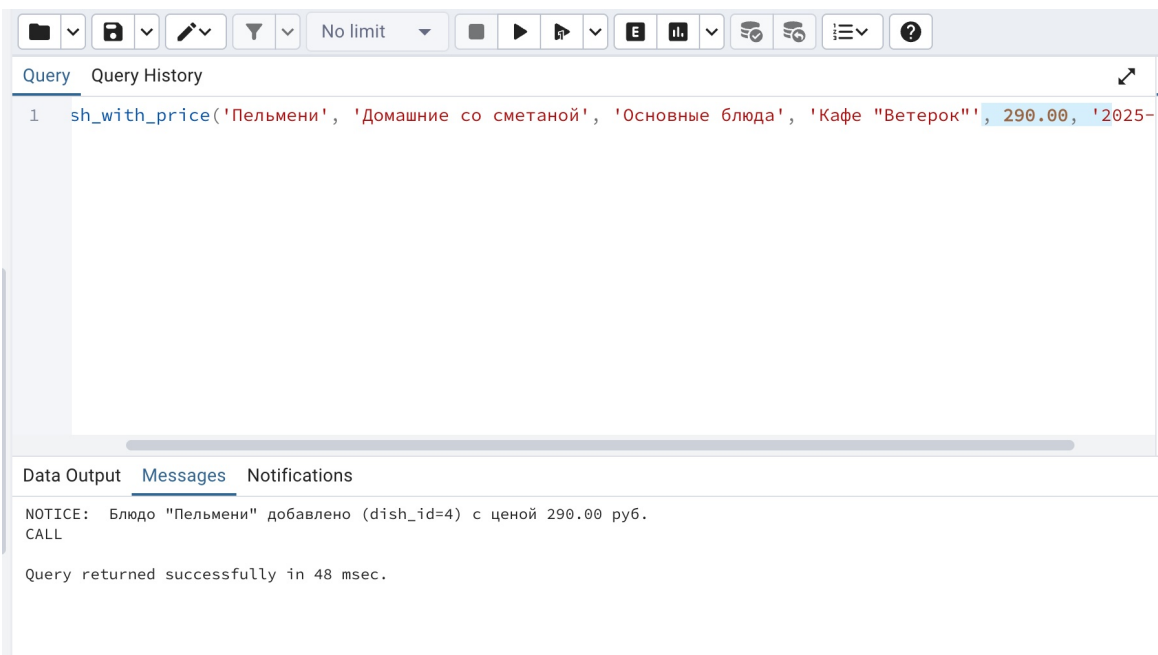
Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 48 msec.

Рисунок 7 — Создание процедуры `add_dish_with_price`

Тест корректного вызова: добавлено блюдо «Пельмени» с начальной ценой 290 руб. в Кафе «Ветерок». Процедура вывела NOTICE с присвоенным `dish_id=4`.



```
Query Query History
1  sh_with_price('Пельмени', 'Домашние со сметаной', 'Основные блюда', 'Кафе "Ветерок"', 290.00, '2025-
```

Data Output Messages Notifications

NOTICE: Блюдо "Пельмени" добавлено (dish_id=4) с ценой 290.00 руб.
CALL

Query returned successfully in 48 msec.

Рисунок 8 — Успешное добавление блюда с ценой через процедуру `add_dish_with_price`

Тест ошибочного вызова: при попытке передать отрицательную цену (-100 руб.) процедура корректно выбросила исключение «Цена не может быть отрицательной».



The screenshot shows a PostgreSQL query editor interface. The top bar has tabs for 'Query', 'Query History', and 'Scratch Pad'. The 'Query' tab is active, showing a single query line: `CALL add_dish_with_price('Тест', 'Тест', 'Основные блюда', 'Летний киоск', -100.00, '2025-01-01');`. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected, displaying an error message:
ERROR: Цена не может быть отрицательной (получено: -100.00)
CONTEXT: PL/pgSQL function add_dish_with_price(character varying,text,character varying,character varying,numeric,date) line 24 at RAISE
SQL state: P0001

Рисунок 9 — Процедура `add_dish_with_price` блокирует ввод отрицательной цены

3.2. Триггеры

Триггер — хранимая процедура, вызываемая автоматически при наступлении событий INSERT, UPDATE или DELETE на заданной таблице. Триггерная функция в PostgreSQL создаётся отдельно и возвращает тип TRIGGER. Для каждого триггера ниже приведены: скрипт создания, тест на корректных данных и тест на некорректных данных.

3.2.1. Триггер `trg_check_inventory_quantity` — запрет отрицательного количества инвентаря

Тип: BEFORE INSERT OR UPDATE на таблице `inventory`, FOR EACH ROW.

Назначение: защита целостности данных — количество единиц инвентаря не может быть отрицательным. При попытке вставить или обновить запись с `quantity < 0` триггер выбрасывает исключение и отменяет операцию.

The screenshot shows a database query editor with a 'Query' tab selected. The query is a PL/pgSQL script to create a trigger. The script is as follows:

```
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF NEW.quantity < 0 THEN
5         RAISE EXCEPTION 'Количество инвентаря не может быть отрицательным. Получено: %', NEW.quar
6     END IF;
7     RETURN NEW;
8 END;
9 $$ LANGUAGE plpgsql;
10
11 CREATE TRIGGER trg_check_inventory_quantity
12 BEFORE INSERT OR UPDATE ON inventory
13 FOR EACH ROW
14 EXECUTE PROCEDURE fn_check_inventory_quantity();
```

Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected, showing the following output:

```
CREATE TRIGGER
Query returned successfully in 47 msec.
```

Рисунок 10 — Создание триггера `trg_check_inventory_quantity`

Тест на некорректных данных (`quantity = -5`): триггер сработал до выполнения `INSERT` и заблокировал вставку с сообщением об ошибке.

The screenshot shows a database query editor with a 'Query' tab selected. The query is an `INSERT` statement:

```
1 INSERT INTO inventory (name, quantity, object_id) VALUES ('Тест', -5, 7);
```

Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected, showing the following error message:

```
ERROR: Количество инвентаря не может быть отрицательным. Получено: -5
CONTEXT: PL/pgSQL function fn_check_inventory_quantity() line 4 at RAISE
SQL state: P0001
```

Рисунок 11 — Триггер блокирует вставку с отрицательным количеством

Тест на корректных данных (`quantity = 5`): вставка прошла без вмешательства триггера, сообщение `INSERT 0 1` подтверждает успешное добавление строки.



The screenshot shows a database query window with two tabs: "Query" and "Query History". The "Query" tab is active, displaying a single SQL statement: `INSERT INTO inventory (name, quantity, object_id) VALUES ('Тест', 5, 7);`. Below the query, there are three tabs: "Data Output", "Messages", and "Notifications". The "Messages" tab is active, showing the output: `INSERT 0 1` and a status message: `Query returned successfully in 48 msec.`

Рисунок 12 — Триггер пропускает корректную вставку

3.2.2. Триггер `trg_set_default_assignment_date` — автоматическая дата назначения

Тип: BEFORE INSERT на таблице `employment_history`, FOR EACH ROW.

Назначение: если при добавлении записи о трудоустройстве поле `assignment_date` не указано (NULL), триггер автоматически подставляет текущую дату. Это исключает ошибки при работе с таблицей без явного указания даты.



The screenshot shows a database query window with two tabs: "Query" and "Query History". The "Query" tab is active, displaying a multi-line SQL script. The script starts with `RETURNS TRIGGER AS $$`, followed by a `BEGIN` block containing an `IF` statement that checks if `NEW.assignment_date` is NULL. If it is, it sets `NEW.assignment_date := CURRENT_DATE;` and raises a notice: `RAISE NOTICE 'Дата назначения не указана — установлена текущая: %', CURRENT_DATE;`. The `IF` block ends with `END IF;` and `RETURN NEW;`. The script ends with `END;` and `$$ LANGUAGE plpgsql;`. Below the script, there are three tabs: "Data Output", "Messages", and "Notifications". The "Messages" tab is active, showing the output: `CREATE TRIGGER` and a status message: `Query returned successfully in 48 msec.`

Рисунок 13 — Создание триггера `trg_set_default_assignment_date`

Тест 1 — без указания даты (`assignment_date = NULL`): триггер автоматически подставил текущую дату 2026-05-22 и вывел NOTICE.



Рисунок 14 — Триггер автоматически устанавливает текущую дату

Тест 2 — с явно указанной датой '2024-03-01': триггер не вмешался в данные, INSERT прошёл без NOTICE.



Рисунок 15 — Триггер не изменяет явно указанную дату

3.2.3. Триггер `trg_prevent_park_delete` — запрет удаления парка с объектами

Тип: BEFORE DELETE на таблице `park`, FOR EACH ROW.

Назначение: обеспечение ссылочной целостности на уровне бизнес-логики. Если в парке есть хотя бы один объект (`park_object`), его удаление блокируется с сообщением о количестве существующих объектов.

The screenshot shows a database query editor with two tabs: "Query" and "Query History". The "Query" tab is active, displaying a PL/pgSQL function definition for a trigger. The code is as follows:

```
1 CREATE OR REPLACE FUNCTION fn_prevent_park_delete()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     v_count INT;
5 BEGIN
6     SELECT COUNT(*) INTO v_count
7     FROM park_object
8     WHERE park_id = OLD.park_id;
9
10    IF v_count > 0 THEN
11        RAISE EXCEPTION 'Нельзя удалить парк (park_id=%), пока в нём есть % объект(ов).', OLD.park_id, v_count;
12    END IF;
13
```

Below the code editor, there are three tabs: "Data Output", "Messages", and "Notifications". The "Messages" tab is active, showing an error message:

```
ERROR: trigger "trg_prevent_park_delete" for relation "park" already exists
SQL state: 42710
```

Рисунок 16 — Создание триггера `trg_prevent_park_delete`

Тест на некорректных данных — попытка удалить парк с объектами (`park_id=1`): триггер заблокировал операцию, сообщив о наличии 2 объектов в парке.

The screenshot shows a database query editor with two tabs: "Query" and "Query History". The "Query" tab is active, displaying a SQL statement:

```
1 DELETE FROM park WHERE park_id = 1;
```

Below the code editor, there are three tabs: "Data Output", "Messages", and "Notifications". The "Messages" tab is active, showing an error message:

```
ERROR: Нельзя удалить парк (park_id=1), пока в нём есть 2 объект(ов).
CONTEXT: PL/pgSQL function fn_prevent_park_delete() line 10 at RAISE
SQL state: P0001
```

Рисунок 17 — Триггер блокирует удаление парка с объектами

Тест на корректных данных — для проверки создан пустой тестовый парк (`park_id=3`):

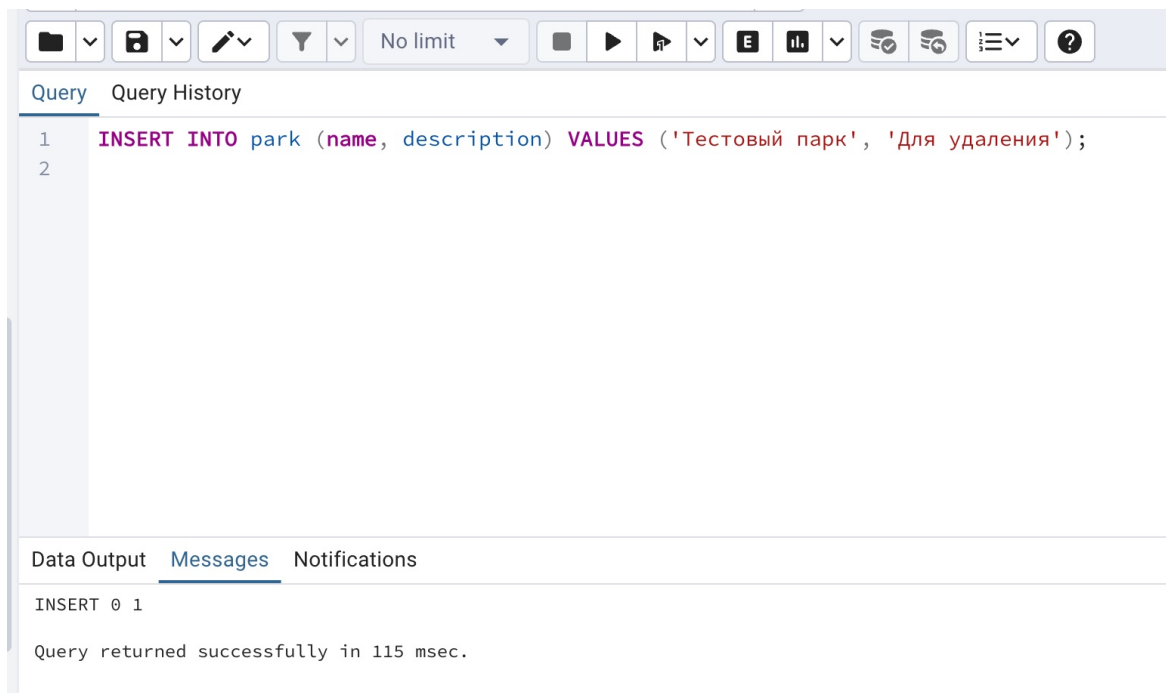


Рисунок 18 — Создание тестового парка для проверки удаления

Удаление пустого парка прошло успешно — триггер не вмешался, в таблице остался только Приморский парк.

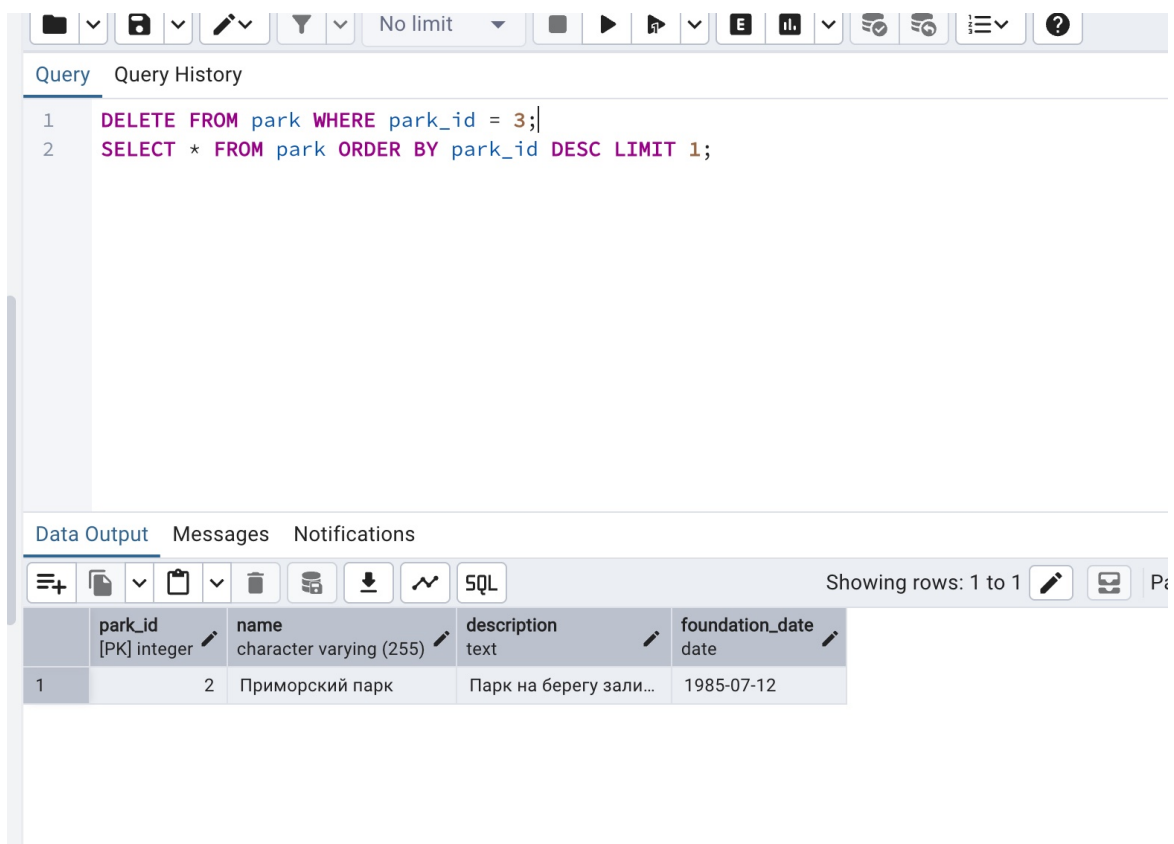
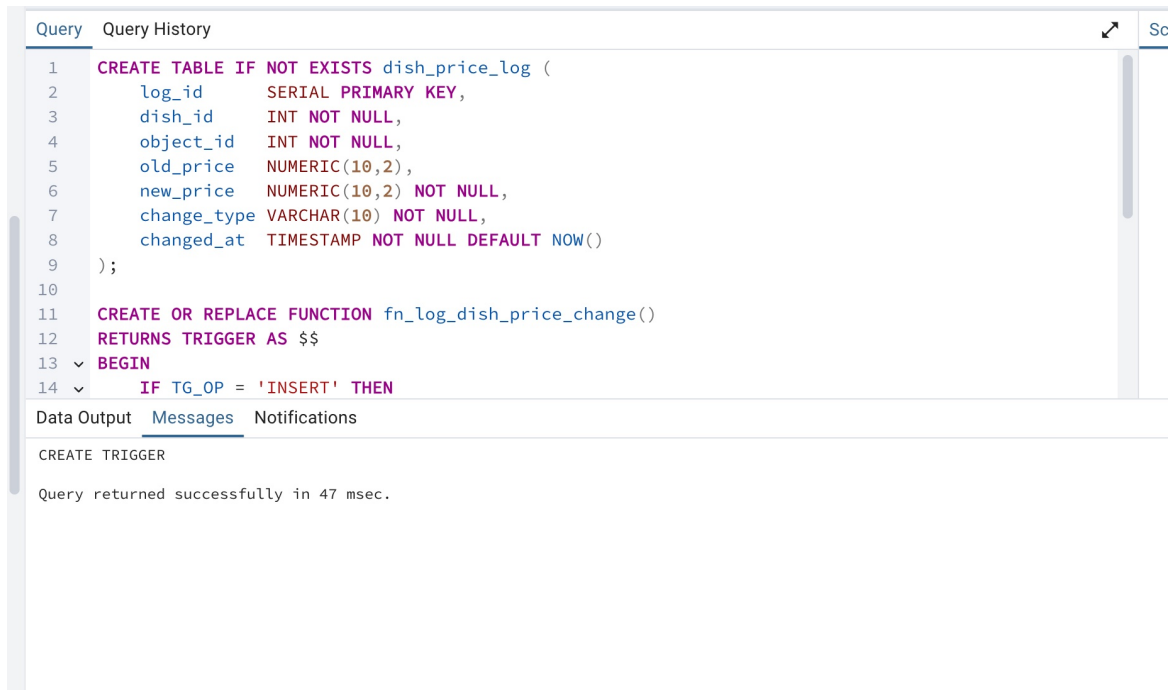


Рисунок 19 — Триггер пропускает удаление парка без объектов

3.2.4. Триггер `trg_log_dish_price_change` — логирование изменений цен

Тип: AFTER INSERT OR UPDATE на таблице dish_price, FOR EACH ROW.

Назначение: автоматический аудит изменений цен на блюда. При каждом добавлении новой цены (INSERT) или её изменении (UPDATE) триггер записывает в таблицу dish_price_log: идентификатор блюда, объекта, старую и новую цену, тип операции и время изменения.



```
1 CREATE TABLE IF NOT EXISTS dish_price_log (  
2     log_id      SERIAL PRIMARY KEY,  
3     dish_id     INT NOT NULL,  
4     object_id   INT NOT NULL,  
5     old_price   NUMERIC(10,2),  
6     new_price   NUMERIC(10,2) NOT NULL,  
7     change_type VARCHAR(10) NOT NULL,  
8     changed_at  TIMESTAMP NOT NULL DEFAULT NOW()  
9 );  
10  
11 CREATE OR REPLACE FUNCTION fn_log_dish_price_change()  
12 RETURNS TRIGGER AS $$  
13 BEGIN  
14     IF TG_OP = 'INSERT' THEN
```

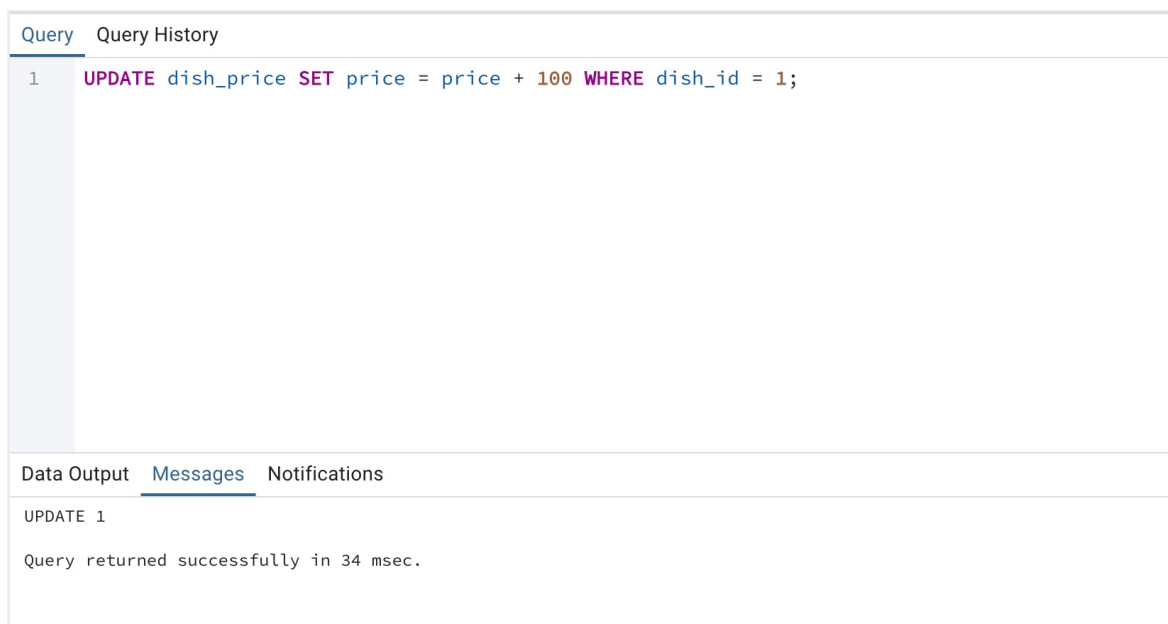
Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 47 msec.

Рисунок 20 — Создание таблицы dish_price_log, функции и триггера

Тест 1 — UPDATE цены блюда (dish_id=1, +100 руб.):



```
1 UPDATE dish_price SET price = price + 100 WHERE dish_id = 1;
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 34 msec.

Рисунок 21 — Выполнение UPDATE для проверки логирования

В таблице dish_price_log появилась запись с типом UPDATE: old_price=350.00, new_price=450.00, время изменения 2026-05-22 02:23.

Query Query History

1 SELECT * FROM dish_price_log;

Data Output Messages Notifications

SQL

Showing rows: 1 to 1

Page No: 1

	log_id [PK] integer	dish_id integer	object_id integer	old_price numeric (10,2)	new_price numeric (10,2)	change_type character varying (10)	changed_at timestamp without time zone
1	1	1	7	350.00	450.00	UPDATE	2026-05-22 02:23:30.6174

Рисунок 22 — Триггер записал UPDATE цены в лог

Тест 2 — INSERT новой цены (dish_id=2 в объекте 7, цена 999 руб.):

Query Query History

1 INSERT INTO dish_price (dish_id, object_id, assignment_date, price)
2 VALUES (2, 7, '2026-01-01', 999.00);

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 48 msec.

Рисунок 23 — Вставка новой цены

В логе появилась вторая запись типа INSERT с old_price=null (значение не существовало ранее) и new_price=999.00.

Query

Query History

1

SELECT * FROM dish_price_log;

<

Рисунок 24 — Триггер записал INSERT новой цены в лог

Итоговое состояние таблицы лога после нескольких UPDATE операций (увеличение цены dish_id=2 на 50 руб.) показывает три записи, отражающие историю изменений по разным объектам.

Query

Query History

1

UPDATE dish_price SET price = price + 50 WHERE dish_id = 2;

2

3

SELECT * FROM dish_price_log ORDER BY log_id DESC LIMIT 3;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 3

Page No: 1

	log_id [PK] integer	dish_id integer	object_id integer	old_price numeric (10,2)	new_price numeric (10,2)	change_type character varying (10)	changed_at timestamp without time zone
1	5	2	7	999.00	1049.00	UPDATE	2026-05-22 02:36:32.177873
2	4	2	9	287.50	337.50	UPDATE	2026-05-22 02:36:32.177873
3	3	2	7	264.50	314.50	UPDATE	2026-05-22 02:36:32.177873

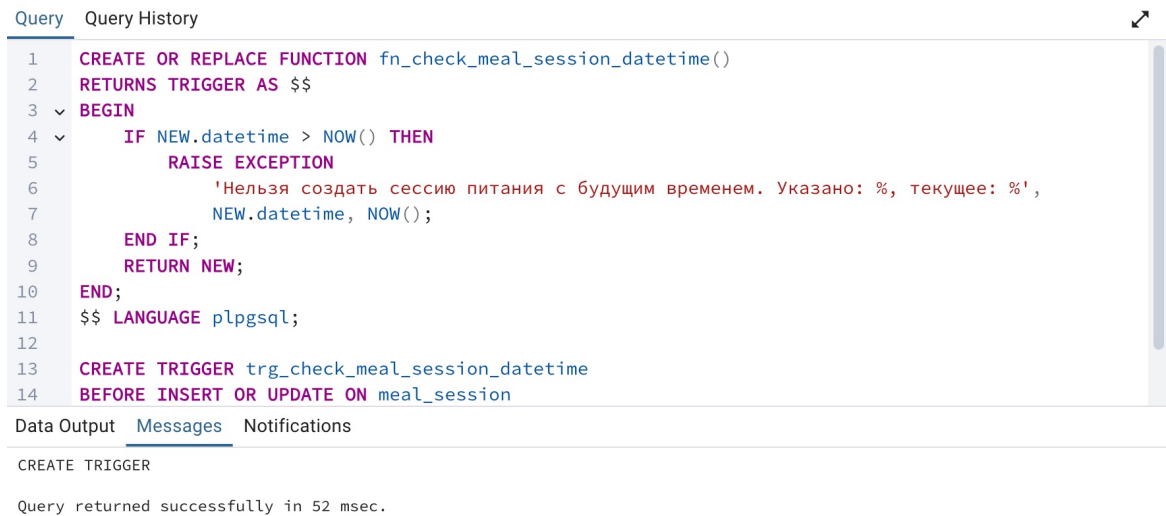
Рисунок 25 — Итоговое состояние таблицы dish_price_log

3.2.5. Триггер trg_check_meal_session_datetime — запрет сессии питания в будущем

Тип: BEFORE INSERT OR UPDATE на таблице meal_session, FOR EACH ROW.

Назначение: контроль корректности временных данных. Сессия питания не может быть создана с датой и временем, превышающими текущий момент (NOW()). При

нарушении условия триггер выбрасывает исключение с указанием переданного и текущего времени.



```
1 CREATE OR REPLACE FUNCTION fn_check_meal_session_datetime()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF NEW.datetime > NOW() THEN
5         RAISE EXCEPTION
6             'Нельзя создать сессию питания с будущим временем. Указано: %, текущее: %',
7             NEW.datetime, NOW();
8     END IF;
9     RETURN NEW;
10 END;
11 $$ LANGUAGE plpgsql;
12
13 CREATE TRIGGER trg_check_meal_session_datetime
14 BEFORE INSERT OR UPDATE ON meal_session
```

CREATE TRIGGER

Query returned successfully in 52 msec.

Рисунок 26 — Создание триггера `trg_check_meal_session_datetime`

Тест на некорректных данных (дата 2099-01-01): триггер заблокировал вставку, сообщив о невозможности создания сессии с будущим временем.



```
1 INSERT INTO meal_session (datetime, object_id, person_id)
2 VALUES ('2099-01-01 12:00:00', 7, 1);
```

ERROR: Нельзя создать сессию питания с будущим временем. Указано: 2099-01-01 12:00:00, текущее: 2026-05-22 02:25:29.55217+03
CONTEXT: PL/pgSQL function fn_check_meal_session_datetime() line 4 at RAISE

SQL state: P0001

Рисунок 27 — Триггер блокирует создание сессии с будущей датой

Тест на корректных данных (`datetime = NOW()`): вставка прошла успешно, триггер не вмешался.



Рисунок 28 — Триггер пропускает сессию с текущим временем

4. Итоговая таблица созданных объектов

Объект	Таблица	Назначение	Результат
add_employee	person, employment_history	Добавление нового сотрудника	✓
dismiss_employee	employment_history	Увольнение сотрудника	✓
add_dish_with_price	dish, dish_price	Добавление блюда с ценой	✓
trg_check_inventory_quantity	inventory	Запрет отрицательного количества	✓
trg_set_default_assignment_date	employment_history	Авто-дата назначения	✓
trg_prevent_park_delete	park	Запрет удаления парка с объектами	✓
trg_log_dish_price_change	dish_price	Логирование изменений цен	✓
trg_check_meal_session_datetime	meal_session	Запрет сессии в будущем	✓

5. Выводы

В ходе выполнения лабораторной работы были изучены и практически реализованы механизмы хранимых процедур и триггеров в СУБД PostgreSQL на базе данных «Легендарный парк».

Были созданы три хранимые процедуры: `add_employee` для добавления нового сотрудника с проверкой существования должности и объекта парка; `dismiss_employee` для увольнения сотрудника с проверкой его активности; `add_dish_with_price` для добавления блюда с одновременной установкой начальной цены и проверкой её корректности.

Были разработаны пять оригинальных триггеров: `trg_check_inventory_quantity` обеспечивает запрет отрицательного количества инвентаря; `trg_set_default_assignment_date` автоматически подставляет текущую дату при её отсутствии; `trg_prevent_park_delete` защищает от удаления парков, содержащих объекты; `trg_log_dish_price_change` ведёт автоматический журнал всех изменений цен на блюда; `trg_check_meal_session_datetime` исключает создание сессий питания с будущим временем.

Работоспособность всех объектов подтверждена тестами на корректных и некорректных входных данных в среде pgAdmin 4.